

Representación en memoria y estructuras lineales

TC-CH04 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Dos programas guardan el mismo millón de lecturas y hacen las mismas operaciones sobre ellas. Uno tarda un segundo y el otro veinte minutos. ¿Qué decidió la diferencia, si el procesador es el mismo?

Producto final. Una pila y una cola escritas por usted con arrays de Bash, capaces de validar el anidamiento de una estructura secundaria de ARN donde un contador falla, y de procesar lecturas respetando el orden de llegada.

Mapa del tema

1. Por qué una estructura de datos decide el éxito de un programa	1
2. Representar un valor antes de organizar una colección	2
3. Modelos de almacenamiento en memoria: contiguo frente a enlazado	4
4. El array y los arrays dinámicos	5
5. La lista enlazada	6
6. La pila (<i>Stack</i>) — Disciplina LIFO	8
7. La cola (<i>Queue</i>) — Disciplina FIFO	10
8. La operación peligrosa: mutación concurrente e índices inválidos	11
9. Actividad de depuración: diagnóstico de errores en punteros	11
10. Síntesis operativa y matriz comparativa de costes	12
11. Glosario conceptual	13
12. Conexión con el curso y siguientes pasos	13
13. Fuentes y reproducibilidad	14

Cómo usar este tema

Este capítulo baja un nivel respecto al anterior. Allí contábamos operaciones sin preguntarnos dónde estaban los datos; aquí la respuesta a esa pregunta es justamente lo que decide el coste. Se lee en dos mitades. La primera es cómo la máquina guarda un número y por qué eso produce resultados que parecen errores y no lo son. La segunda son las estructuras lineales, array, lista, pila y cola, cada una con las operaciones que hace baratas y las que hace caras.

Al terminar sabrá: explicar por qué una resta puede hacerse con el circuito de la suma, y por qué comparar dos números decimales con el signo igual es mala idea; distinguir un tipo abstracto de su implementación, que es la distinción que permite cambiar una por otra sin tocar el resto del programa; elegir entre almacenamiento contiguo y enlazado según qué operación domine; y reconocer los dos problemas que resuelve una pila y los que resuelve una cola.

La ruta **esencial** son la representación en memoria, el par contiguo frente a enlazado, y la pila y la cola con sus operaciones. La ruta de **estudio** añade el coste amortizado del array dinámico, la lista doblemente enlazada, el caso del ARN y el anexo.

1 Por qué una estructura de datos decide el éxito de un programa

ESENCIAL. Resolver un problema algorítmico es solo el primer paso; hacerlo viable sobre conjuntos masivos de datos biomédicos es el verdadero reto de la bioinformática. Y esa viabilidad depende mucho más de **cómo se organizan y almacenan los datos en la memoria** que de la velocidad del procesador.

La misma operación básica —buscar un nucleótido o variante, insertar una nueva lectura de secuenciación, o recuperar el siguiente elemento a procesar— puede costar un tiempo constante ($O(1)$) o crecer linealmente con el tamaño de los datos ($O(n)$) según la estructura de datos elegida. Sobre un genoma humano de 3×10^9 pares de bases o un experimento de transcriptómica con millones de moléculas, esa diferencia es la frontera exacta entre una respuesta instantánea y un proceso que no termina en un tiempo útil.

La misma operación sobre una colección de datos —buscar, insertar, recuperar el siguiente— puede costar tiempo constante o crecer linealmente con el tamaño de la colección, según la estructura de datos elegida para guardarla.

Cualquier asistente de inteligencia artificial puede generar código para manipular listas o matrices en segundos, pero casi siempre elegirá la estructura «por defecto» del lenguaje sin considerar las consecuencias en consumo de memoria o latencia de acceso. El objetivo de este capítulo no es simplemente aprender la sintaxis de una lista o una pila, sino comprender los fundamentos físicos de la memoria, qué garantías ofrece cada estructura y a qué precio computacional, para elegir siempre con criterio científico.

2 Representar un valor antes de organizar una colección

Para estructurar grandes colecciones biológicas, es indispensable comprender primero cómo se codifica y almacena un único dato elemental en los circuitos del ordenador.

2.1 Dato, tipo de dato y estructura de datos

- Un **dato** es una unidad mínima de información o valor concreto (por ejemplo, la base nitrogenada 'A' o una temperatura de desnaturalización 95.0).
- Un **tipo de dato** define formalmente el conjunto de valores posibles que puede tomar una variable y el repertorio de operaciones válidas que tienen sentido sobre ellos (no tiene sentido multiplicar dos cadenas de ADN, pero sí concatenarlas).
- Una **estructura de datos** es una forma sistemática de organizar, relacionar y almacenar una colección de datos en la memoria del ordenador, junto con los algoritmos que permiten acceder a ellos y manipularlos de forma eficiente.

Un dato es un valor; un tipo de dato define qué valores son posibles y qué operaciones tienen sentido sobre ellos; una estructura de datos organiza una colección de datos en memoria junto con las operaciones que permite hacer sobre ella eficientemente.

Concepto. Hay un tipo de dato cuyo valor es una dirección de memoria: el *puntero*. No guarda un número ni una letra, guarda dónde vive otra cosa. Conviene verlo como un tipo más y no como un detalle interno, porque es el que hace posible el almacenamiento enlazado: un nodo puede estar en cualquier hueco libre de la memoria, por lejos que quede de su vecino, porque el vecino guarda su dirección. Todo lo que viene después en este capítulo depende de esa idea.

2.2 Codificación física en memoria: ceros y unos

En el soporte físico de la memoria RAM, toda la información se reduce a estados binarios: bits (0 y 1). Interpretar esos bits exige conocer su esquema de codificación:

1. **Enteros en base 2:** los números naturales se representan en base binaria pura (por ejemplo, el valor decimal 115 se codifica en 8 bits como 01110011).
2. **Enteros con signo (Complemento a dos):** en complemento a dos no se reserva un bit de signo independiente: para una palabra de w bits, el bit más significativo (b_{w-1}) aporta un peso negativo -2^{w-1} y los restantes bits aportan pesos positivos estándar. Así, en 8 bits, 11111111 representa $-128 + 127 = -1$, mientras que 01111111 representa $+127$. Este esquema genera el intervalo asimétrico $[-2^{w-1}, 2^{w-1} - 1]$ y permite que la suma y la resta utilicen el mismo circuito binario.
3. **Coma flotante (Floating Point):** los números reales se codifican siguiendo el estándar internacional IEEE 754 como el producto de un bit de signo, un significando (o mantisa) y una base elevada a un exponente ($v = (-1)^s \times M \times 2^E$). Ocupan habitualmente 32 bits (precisión simple) o 64 bits (doble precisión).
4. **Caracteres y texto:** se representan asignando un número entero único a cada símbolo gráfico. El estándar ASCII clásico emplea 7 bits (128 caracteres básicos del alfabeto inglés). El estándar Unicode (con esquemas como UTF-8 de longitud variable de 1 a 4 bytes) permite representar más de 140 000 símbolos universales, alfabetos globales y caracteres técnicos.

En complemento a dos, que es lo que usan los procesadores actuales, no hay un bit de signo aparte: el bit más significativo pesa -2^{w-1} y los demás pesan en positivo. De ahí sale el rango asimétrico de -128 a 127 en ocho bits, y de ahí que la resta pueda hacerse con el mismo circuito que la suma. Los números con decimales se representan como un coeficiente por una potencia de dos, con precisión limitada, y los caracteres asignando un número a cada símbolo.

2.3 Mecanismo del complemento a dos

El complemento a dos es uno de los logros más elegantes de la ingeniería computacional: permite a la CPU realizar restas utilizando exactamente los mismos circuitos lógicos de la suma.

Para obtener la representación de -115 en 8 bits:

1. Se parte del valor positivo $+115$ en binario: 01110011 .
2. Se invierten todos los bits (operación *NOT*): 10001100 .
3. Se suma 1 al resultado: 10001101 .

La verificación matemática es inmediata al sumar $+115$ y -115 :

$$\begin{array}{rcl} & 01110011 & (+115) \\ + & 10001101 & (-115) \\ \hline = & 10000000 & (256 \equiv 0 \text{ (mód 256)}) \end{array}$$

Al descartar el noveno bit (el bit de acarreo que excede la capacidad del registro de 8 bits), el resultado es 00000000 , es decir, 0 exacto.

Representar -115 en complemento a dos de 8 bits: invertir los bits de 115 ($01110011 \rightarrow 10001100$) y sumar 1 (10001101). Sumar 01110011 (115) y 10001101 (el resultado) da 100000000 ; descartando el noveno bit (el acarreo que excede los 8 bits) queda 00000000 , cero, confirmando que la representación es correcta: $115 + (-115) = 0$.

Se reproduce con `verification/chapter_results.sh twos_complement 115 8`.

2.4 La trampa de la coma flotante: imprecisión binaria

En coma flotante, muchas fracciones decimales cotidianas (como $0.1 = 1/10$ o $0.2 = 2/10$) **no tienen una representación binaria finita**, de la misma forma que la fracción $1/3 = 0.3333\dots$ no tiene fin en el sistema decimal.

Al sumar $0.1 + 0.2$ en coma flotante de doble precisión (64 bits), el valor almacenado es:

$$0.1 + 0.2 = 0.3000000000000000444089209850062616169452667236328125$$

Por esta razón, una comparación directa de igualdad estricta ($0.1 + 0.2 == 0.3$) evaluará como **FALSA**. En computación científica y bioinformática, los valores decimales nunca deben compararse con igualdad estricta, sino mediante una tolerancia o épsilon: $|a - b| < \epsilon$.

En coma flotante de doble precisión, $0.1 + 0.2$ no da exactamente 0.3 : el resultado es 0.30000000000000004 , distinto de 0.3 en el último dígito representable, porque ni 0.1 ni 0.2 tienen representación binaria finita. Comparar directamente el resultado de una suma de decimales con `==` es, por esta razón, un error frecuente en código generado tanto a mano como por una IA.

Se reproduce con `verification/chapter_results.sh float_imprecision`.

2.5 Tipos de Datos Abstractos (TDA) frente a Implementación

Una distinción fundamental en informática es la separación entre el **qué** y el **cómo**:

- Un **Tipo de Dato Abstracto (TDA)** define el comportamiento conceptual externo: qué operaciones ofrece (por ejemplo, «insertar al final», «extraer el más antiguo») y qué reglas semánticas cumplen, sin especificar en absoluto cómo se almacenan los datos por dentro.

- La **Implementación** es la realización concreta en código y memoria (usando un vector contiguo o nodos enlazados).

El TDA establece el contrato funcional; la implementación determina el coste en tiempo y memoria de cada operación.

Un tipo de dato abstracto (TDA) especifica qué operaciones ofrece una estructura y qué significan, sin fijar cómo se implementa; la implementación es la realización concreta que determina el rendimiento. Dos implementaciones del mismo TDA ofrecen las mismas operaciones con costes distintos.

3 Modelos de almacenamiento en memoria: contiguo frente a enlazado

ESENCIAL

Existen dos estrategias físicas fundamentales para colocar una colección de elementos en la memoria RAM:

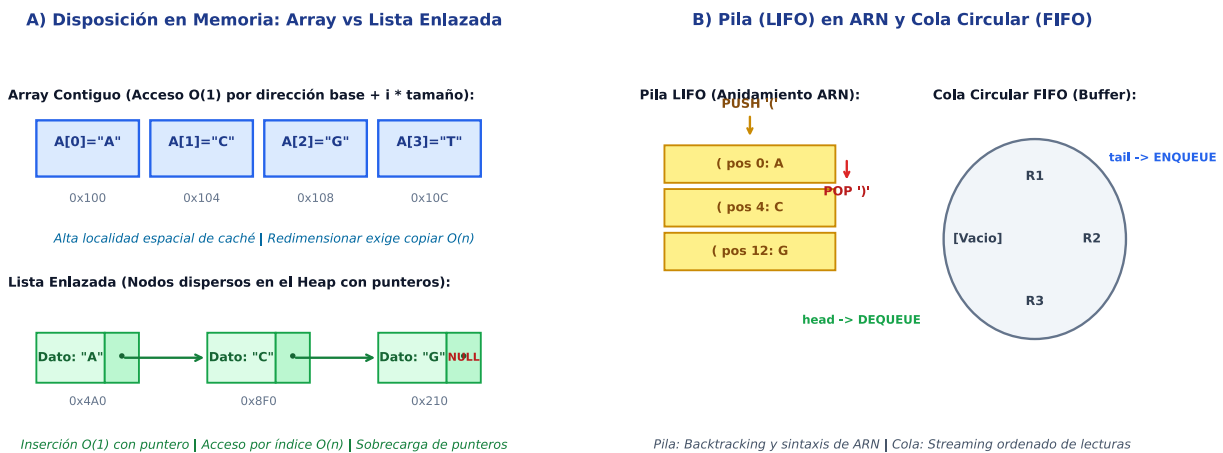


Figura 1: A) Comparación física en memoria: Array contiguo con cálculo aritmético directo $O(1)$ vs Lista Enlazada con nodos dispersos en Heap y punteros $O(n)$. B) Estructuras lineales especializadas: Pila LIFO (anidamiento de pares de bases de ARN) y Cola Circular FIFO (buffer de lecturas con punteros head/tail). Figura original adaptada de las presentaciones docentes.

```
Almacenamiento Contiguo (Array, base = 1000, elemento = 4 bytes):
Direccion: 1000 1004 1008 1012 1016
+-----+-----+-----+-----+
| e[0] | e[1] | e[2] | e[3] | e[4] |
+-----+-----+-----+-----+

Formula de acceso: dir(e[i]) = base + i * tamaño ->  $O(1)$  directo

Almacenamiento Enlazado (Nodos dispersos en memoria):
Dir 3040          Dir 1592          Dir 4801
+-----+-----+ +-----+-----+ +-----+-----+
| val0 | 1592 | --> | val1 | 4801 | --> | val2 | null |
+-----+-----+ +-----+-----+ +-----+-----+
(Sin relacion aritmetica: para llegar a val2 hay que recorrer val0 y val1)
```

Figura 2: Comparación entre almacenamiento contiguo (array) y enlazado (lista). En el array, la dirección física se calcula inmediatamente en $O(1)$ mediante una fórmula aritmética. En la lista enlazada, las direcciones son arbitrarias y se deben seguir los punteros secuencialmente en $O(n)$. Figura original.

El almacenamiento contiguo coloca los elementos en direcciones consecutivas, separadas por el tamaño de cada elemento, así que la dirección de cualquier elemento se calcula directamente desde su índice; el almacenamiento

enlazado coloca cada nodo en cualquier dirección libre de memoria, sin relación aritmética entre las direcciones de nodos sucesivos, y la única forma de encontrar el siguiente nodo es seguir el puntero que guarda.

3.1 Cálculo de dirección directa en almacenamiento contiguo

En un array, si la dirección base es 1000 y cada elemento ocupa 4 bytes, la dirección del elemento con índice i se calcula con una única multiplicación y una suma:

$$\text{Dirección}(e[i]) = \text{base} + i \times \text{tamaño}$$

Para el índice 7: $1000 + 7 \times 4 = 1028$. Para el índice 999: $1000 + 999 \times 4 = 4996$. Ambas operaciones toman exactamente el mismo tiempo: **coste constante** $O(1)$.

En cambio, en una lista enlazada no existe ninguna relación matemática entre las direcciones de los nodos. Para acceder al nodo 999 es estrictamente obligatorio recorrer los 998 nodos precedentes saltando de puntero en puntero: **coste lineal** $O(n)$.

En un array de enteros de 4 bytes que empieza en la dirección 1000, la dirección del elemento de índice i es $1000 + i \times 4$ sin recorrer nada: el elemento 7 está en 1028 y el elemento 999 está en 4996, ambos con una sola multiplicación y una suma. En una lista enlazada no existe esa fórmula: alcanzar el elemento i exige i saltos de puntero, uno por cada nodo intermedio.

Se reproduce con `verification/chapter_results.sh array_address_calc 1000 4 999`.

3.2 Costes fundamentales del modelo de memoria

- **Leer o escribir en una dirección:** tiempo constante $O(1)$.
- **Alojar un bloque contiguo de tamaño N :** tiempo $O(N)$, pues el sistema operativo debe localizar un segmento contiguo libre y reservarlo.
- **Inmutabilidad del bloque contiguo:** un bloque alojado no puede expandirse «en el mismo sitio» si la celda contigua ya está ocupada por otra variable. Para crecer, es obligatorio alojar un nuevo bloque más grande en otra zona de la memoria y copiar todos los datos existentes.
- **Liberar memoria (*free*):** tiempo constante $O(1)$.

En el almacenamiento contiguo, acceder al elemento i cuesta tiempo constante por cálculo directo de dirección, pero insertar en medio obliga a desplazar los elementos posteriores; en el almacenamiento enlazado, insertar o quitar es solo recolocar punteros sin mover datos, pero no hay acceso directo y hay que recorrer la cadena desde el principio.

Una estructura estática tiene tamaño fijado de antemano; una dinámica crece y mengua durante la ejecución. Las estructuras dinámicas son las más usadas en bioinformática porque el tamaño de los datos rara vez se conoce por adelantado.

Leer o escribir en una dirección de memoria cuesta $O(1)$; alojar un bloque contiguo de tamaño N cuesta $O(N)$ y, una vez alojado, no se expande en el mismo sitio: hace falta alojar un bloque mayor y copiar los datos; desalojar memoria cuesta $O(1)$.

Modelo docente de coste de memoria.

En este capítulo modelamos formalmente como $O(n)$ el coste de inicializar explícitamente n celdas contiguas de memoria o copiar n elementos durante el redimensionado de un array dinámico. Las solicitudes y liberaciones virtuales al sistema operativo pueden tener comportamientos dependientes del asignador y del lenguaje, pero la cota $O(n)$ del array dinámico se fundamenta invariablemente en la necesidad de copiar los n elementos preexistentes.

4 El array y los arrays dinámicos

ESENCIAL

El array es la estructura contigua por excelencia. Ofrece acceso directo $O(1)$ por índice, pero penaliza severamente

las inserciones o eliminaciones intermedias ($O(n)$), ya que todos los elementos posteriores deben desplazarse una posición a la derecha o a la izquierda.

El acceso por índice a un array cuesta $O(1)$; la búsqueda de un valor cuesta $O(n)$ si no está ordenado, u $O(\log n)$ con búsqueda binaria si está ordenado; la inserción o el borrado en una posición intermedia cuesta $O(n)$ porque hay que desplazar los elementos posteriores.

4.1 Redimensionamiento geométrico y coste amortizado

En la práctica bioinformática no se conoce de antemano cuántas lecturas o variantes se van a procesar. Para evitar fijar un tamaño estático, los lenguajes modernos implementan **arrays dinámicos** (vectores con crecimiento automático).

¿Cómo crece un array dinámico eficientemente?

- Si incrementara su capacidad de 1 en 1 cada vez que se añade un elemento, insertar N elementos requeriría copiar $1 + 2 + 3 + \dots + N = O(N^2)$ elementos en total: una ineficiencia inasumible.
- En su lugar, utiliza una **estrategia de duplicación geométrica**: cuando el array se llena (capacidad C), se aloja un nuevo bloque de tamaño $2C$ y se copian los elementos existentes.

Aunque la inserción puntual que dispara la duplicación cuesta $O(N)$, ocurre tan raramente que el coste total para insertar N elementos es:

$$N + \left(\frac{N}{2} + \frac{N}{4} + \dots + 4 + 2 + 1 \right) < N + N = 2N \Rightarrow O(N) \text{ en total}$$

Dividiendo el coste total entre las N operaciones, el coste medio por inserción es $\frac{2N}{N} = 2 = O(1)$ **amortizado**.

Un array dinámico que duplica su capacidad cada vez que se llena logra que añadir N elementos al final cueste en total $O(N)$ operaciones de copia, es decir $O(1)$ amortizado por elemento, frente a las $O(N^2)$ que costaría redimensionar de uno en uno.

Se reproduce con `verification/chapter_results.sh dynamic_array_growth 16`.

Para un patrón de acceso aleatorio frecuente sin inserciones ni borrados, un array (acceso $O(1)$) es preferible a una lista enlazada (acceso $O(n)$): sobre una colección de tamaño n , leer m posiciones aleatorias cuesta $O(m)$ con un array frente a $O(m \cdot n)$ en el peor caso con una lista.

Se reproduce con `verification/chapter_results.sh array_vs_list_access 1000`.

Caso límite: Acceso fuera de rango (*Out of Bounds*).

Un array de tamaño n tiene índices válidos estrictamente desde 0 hasta $n - 1$. Intentar acceder al índice n o a un índice negativo carece de sentido. En lenguajes de bajo nivel como C, este acceso no se comprueba, leyendo memoria arbitraria del sistema y generando graves fallos de seguridad o corrupción de datos. En entornos de scripting seguros o lenguajes modernos, se produce una excepción controlada o se devuelve un valor nulo.

Acceder al índice n (uno más allá del último válido, 0 a $n - 1$) o a un índice negativo en un array de n elementos no tiene elemento que devolver. En Bash, leer un índice fuera de rango de un array devuelve una cadena vacía en vez de fallar; en lenguajes sin comprobación de límites como C, el mismo acceso lee memoria ajena sin detectarlo. El comportamiento exacto ante un índice inválido depende del lenguaje y debe comprobarse, no asumirse.

Se reproduce con `verification/chapter_results.sh array_out_of_bounds 5`.

5 La lista enlazada

ESTUDIO

Una lista enlazada almacena sus elementos en **nodos** independientes dispersos en la memoria. Cada nodo contiene dos campos:

- dato: el valor biológico almacenado.

- siguiente: el puntero (dirección física) al nodo posterior en la secuencia.

```
insertarInicio(L, x):
    nuevo <- crearNodo(x)
    nuevo.siguiente <- L.cabeza
    L.cabeza <- nuevo

buscar(L, x):
    actual <- L.cabeza
    mientras actual != null:
        si actual.dato == x:
            devolver actual
        actual <- actual.siguiente
    devolver null
```

La inserción al inicio de una lista enlazada cuesta $O(1)$ porque solo se recolocan dos punteros; el acceso por posición y la búsqueda cuestan $O(n)$ porque no hay atajo al elemento i : hay que recorrer la cadena. El borrado de la cabeza cuesta $O(1)$.

5.1 El orden de asignación de punteros importa

Uno de los errores más comunes al programar listas enlazadas es alterar el orden en que se reasignan los punteros:

```
insertarInicio_ERROR(L, x):
    nuevo <- crearNodo(x)
    L.cabeza <- nuevo           # ERROR: se pierde la referencia a la lista
    nuevo.siguiente <- L.cabeza # El nuevo nodo apunta a si mismo en bucle
```

Si se mueve `L.cabeza` antes de enlazar `nuevo.siguiente`, la dirección de la lista original se pierde irremediablemente en la memoria (*fuga de memoria* o *memory leak*), y el nuevo nodo queda atrapado apuntando a sí mismo en un ciclo infinito.

Si la cabeza se mueve al nodo nuevo antes de enlazar ese nodo con la cabeza anterior, el nodo nuevo acaba apuntándose a sí mismo. Queda un bucle de un solo nodo y el resto de la lista se pierde: nadie guarda ya su dirección.

Se reproduce con `verification/chapter_results.sh linked_list_pointer_bug`.

5.2 Variantes: Listas doblemente enlazadas y circulares

- **Lista doblemente enlazada (*Doubly Linked List*):** cada nodo almacena dos punteros: `siguiente` y `anterior`. Esto permite navegar en ambas direcciones y eliminar un nodo conocido en tiempo $O(1)$ sin tener que buscar quién lo precedía.
- **Lista circular:** el puntero `siguiente` del último nodo apunta de nuevo a la cabeza, formando un anillo continuo.

En una lista doblemente enlazada, cada nodo guarda también un puntero al nodo anterior, lo que permite recorrer hacia atrás y borrar un nodo ya localizado en $O(1)$ sin recorrer la lista de nuevo.

Insertar al final de una lista enlazada simple de n nodos sin puntero de cola cuesta $O(n)$ (recorrer hasta el último); manteniendo un puntero de cola actualizado, la misma inserción cuesta $O(1)$. Pero borrar el último nodo sigue costando $O(n)$ incluso con puntero de cola, porque no hay forma de retroceder desde el último nodo en una lista simplemente enlazada para recolocar el nuevo puntero de cola: solo una lista doblemente enlazada resuelve también el borrado del final en $O(1)$.

Se reproduce con `verification/chapter_results.sh linked_list_tail_insert 10`.

Buscar un valor ausente en una lista enlazada de n nodos recorre los n nodos completos sin encontrar coincidencia y termina en `null` cuando `actual` pasa a ser `null`, no por comparación con el valor buscado. El resultado de

un recorrido completo sin coincidencia debe distinguirse explícitamente de un valor encontrado por accidente; devolver un valor por defecto ambiguo (como 0) en vez de null impide esa distinción.

Se reproduce con `verification/chapter_results.sh linked_list_search_miss 10`.

6 La pila (*Stack*) — Disciplina LIFO

ESENCIAL

Una pila es un TDA lineal restrictivo regido por la regla **LIFO** (*Last In, First Out*): el último elemento en entrar es estrictamente el primero en salir.

Ofrece tres operaciones fundamentales, todas en tiempo constante $O(1)$:

- `push(x)`: coloca el elemento x en la cima de la pila.
- `pop()`: retira y devuelve el elemento situado en la cima.
- `peek()` (o `top()`): consulta el elemento de la cima sin retirarlo.

La necesidad operativa de la pila: retroceso y desapilado.

Más allá de almacenar datos, la pila es la estructura computacional natural para cualquier proceso que deba **des-hacer acciones** o realizar **retroceso exploratorio** (*backtracking*). Cuando un algoritmo recorre una bifurcación y encuentra un camino inválido o un símbolo incompatible, desapilar (`pop`) devuelve inmediatamente el estado computacional al punto de decisión previo en tiempo $O(1)$, sin requerir búsquedas en la memoria.

Una pila ofrece tres operaciones, todas $O(1)$: `push(x)` (poner x en la cima), `pop()` (sacar y devolver el elemento de la cima) y `peek()` (mirar la cima sin sacarla). Hacer `pop()` sobre una pila vacía es un caso límite que debe detectarse, no ejecutarse.

6.1 Casos límite de la pila: Underflow y Overflow

- **Subdesbordamiento** (*Stack Underflow*): intentar ejecutar `pop()` o `peek()` cuando la pila está vacía. Un programa robusto debe comprobar esta condición antes de desapilar.
- **Desbordamiento** (*Stack Overflow*): intentar ejecutar `push()` sobre una pila implementada con capacidad estática que ya está llena. Es la misma causa que el colapso de memoria por recursión infinita, visto en el capítulo de algoritmos y coste.

Hacer `pop()` sobre una pila vacía es un subdesbordamiento (*stack underflow*); hacer `push()` sobre una pila de capacidad fija ya llena es un desbordamiento (*stack overflow*), el mismo nombre y la misma causa de fondo —falta de espacio donde crecer— que el desbordamiento de la pila de llamadas por recursión sin caso base, visto en el capítulo de algoritmos y coste.

Restringir una colección a un único punto de acceso (pila o cola) es una decisión de diseño, no una limitación técnica: al eliminar operaciones como leer o modificar un elemento intermedio se descarta de raíz esa clase de errores y el comportamiento de la estructura queda predecible sin necesidad de rastrear todos sus estados posibles.

Característica	Pila sobre Array Dinámico	Pila sobre Lista Enlazada
Rendimiento push/pop	$O(1)$ amortizado	$O(1)$ garantizado en cada paso
Localidad de caché	Excelente (elementos contiguos)	Pobre (nodos dispersos en RAM)
Sobrecarga de memoria	Baja (solo datos y un índice)	Alta (un puntero extra por nodo)

Una pila implementada sobre un array dinámico ofrece push/pop en $O(1)$ amortizado con excelente localidad de caché y poca sobrecarga por elemento, a cambio de memoria potencialmente desperdiciada y tamaño flexible solo mediante redimensionados costosos; sobre una lista enlazada ofrece $O(1)$ garantizado en cada operación individual y tamaño dinámico sin redimensionados, a cambio de un puntero adicional por nodo y peor localidad de caché.

6.2 Caso bioinformático: Validación de estructura secundaria de ARN

Las estructuras secundarias de ARN (horquillas, bucles y hélices) se anotan mediante la notación de corchetes y puntos (*dot-bracket notation*), donde los pares de bases emparejadas se representan con paréntesis. Conviene decirlo con precisión, porque aquí se confunden dos cosas distintas. En la notación de puntos y paréntesis, un punto es una base sin emparejar y cada par de paréntesis marca dos bases emparejadas entre sí. Cuando dos pares se cruzan, y por tanto no pueden anidarse uno dentro del otro, se recurre a una segunda clase de paréntesis, los corchetes: así se anotan los pseudonudos. Lo que validaremos en este capítulo es que **cada clase esté bien anidada consigo misma**, que es exactamente lo que sabe hacer una pila.

Un enfoque ingenuo para verificar si una cadena está bien balanceada es usar un **contador numérico** que sume +1 al abrir y reste -1 al cerrar. Sin embargo, sobre la cadena "([])", el contador evalúa +1, +2, +1, 0 (nunca negativo y termina en cero), por lo que **la acepta erróneamente**.

Alcance del validador y delimitación formal.

Bajo la gramática libre de contexto que modela este ejercicio, se aceptan exclusivamente pares correctamente anidados con delimitadores independientes () y []. La cadena "([])" se rechaza porque el cierre) no coincide con la apertura [situada en la cima de la pila.

El veredicto formal de rechazo significa estrictamente: «*la cadena no pertenece al lenguaje anidado que valida este autómatas de pila*». No debe interpretarse como «estructura biológicamente imposible», puesto que los pseudonudos reales existen en la naturaleza, pero su modelado estructural requiere gramáticas y formalismos más complejos que una pila elemental.

Una **pila** resuelve el problema de forma exacta:

1. Lee '(': apila '('.
2. Lee '[': apila '['.
3. Lee ')': consulta la cima. La cima contiene '[', que **no coincide** con ')'. La pila **rechaza inmediatamente**.

Símbolo	Contador	Pila (cima der.)	Veredicto Contador	Veredicto Pila
(	+1 = 1	(	Válido (> 0)	Apila (
[	+1 = 2	([	Válido (> 0)	Apila [
)	-1 = 1	(	Válido (> 0)	Rechaza:) no casa con [
]	-1 = 0	—	Termina en 0: acepta, y se equivoca	Ya rechazada

Figura 3: Traza comparativa sobre "([])". El contador acepta por error al no distinguir tipos de apertura; la pila valida estrictamente el emparejamiento por clases. Figura original.

Sobre la entrada "([])", un contador simple de aperturas y cierres la acepta como bien anidada (nunca negativo, termina en 0), pero una pila que compara cada cierre con el símbolo en la cima la rechaza correctamente, porque) no casa con el [que está en la cima cuando aparece.

Se reproduce con `verification/chapter_results.sh stack_validate '([)]'`.

```
ALGORITMO: Pila_Validacion_Estructura_Secundaria_ARN
ENTRADA:
- S: Cadena en notacion dot-bracket de longitud n sobre alfabeto {'(', ')', '[', ']', '.'}
SALIDA:
- Veredicto: {ESTRUCTURA_VALIDA, ERROR_SINTAXIS_DESBALANCEO, ERROR_TIPO_CIERRE_INCORRECTO}
COMPLEJIDAD: Tiempo O(n) un solo recorrido lineal, Espacio auxiliar O(n) para la pila

1. INICIALIZACION:
Pila <- Crear_Pila_Vacia()
Mapa_Pares <- {'(': ')', '[': ']'} # Mapeo de cierre a su apertura esperada

2. RECORRIDO LINEAL DE CARACTERES:
Para pos desde 0 hasta n - 1:
```

```

simbolo <- S[pos]

Si simbolo en {'(', '['} entonces:
    Pila.PUSH((simbolo, pos)) # Guardar tipo de apertura y su coordenada

Sino si simbolo en {'}', ']'} entonces:
    Si Pila.IS_EMPTY() entonces:
        Retornar ERROR_SINTAXIS_DESBALANCEO (Cierre sin apertura previa en pos)

    (cima_simbolo, cima_pos) <- Pila.POP()
    apertura_esperada <- Mapa_Pares[simbolo]

    Si cima_simbolo != apertura_esperada entonces:
        Retornar ERROR_TIPO_CIERRE_INCORRECTO (Cierre 'simbolo' no casa con 'cima_simbolo')

Sino si simbolo == '.' entonces:
    Continuar # Base no emparejada en bucle o extremo terminal

3. COMPROBACION FINAL DE CIERRE:
Si NO Pila.IS_EMPTY() entonces:
    Retornar ERROR_SINTAXIS_DESBALANCEO (Quedaron aperturas sin su correspondiente cierre)

4. RETORNO:
Retornar ESTRUCTURA_VALIDA

```

7 La cola (*Queue*) — Disciplina FIFO

ESENCIAL

Una cola es un TDA lineal regido por la disciplina **FIFO** (*First In, First Out*): el primer elemento que entra es el primero en ser procesado, como una cola de impresión o una tubería física.

Ofrece dos operaciones maestras en tiempo $O(1)$:

- `enqueue(x)`: inserta el elemento x al final (*rear* o *tail*) de la cola.
- `dequeue()`: retira y devuelve el elemento situado en el frente (*front* o *head*).

Y dos consultas que no la modifican, tan necesarias como las anteriores: *frente*, también llamada *peek*, que devuelve el elemento que saldría a continuación sin sacarlo, y *esta_vacia*, que responde si queda algo dentro. La segunda es la que evita el subdesbordamiento: se pregunta antes de sacar, no se saca y se comprueba después. La pila tiene exactamente las mismas dos consultas con otros nombres.

Una cola ofrece `enqueue(x)` (añadir x al final) y `dequeue()` (sacar y devolver el elemento del frente), ambas $O(1)$ con una implementación adecuada.

7.1 Colas circulares sobre arrays (Buffers de anillo)

Si una cola se implementa sobre un array estático simple avanzando el frente y el final, tras varios encolados y desencolados el puntero final llegará al tope del array, desaprovechando los huecos libres dejados al inicio.

Una **cola circular** (*ring buffer*) soluciona este problema mediante **aritmética modular**:

$$\text{nuevo_indice} = (\text{indice} + 1) \bmod M$$

Al llegar al extremo derecho del array de capacidad M , el puntero «da la vuelta» (*wrap-around*) y reutiliza las posiciones libres del principio sin necesidad de desplazar ningún dato existente.

Una cola circular reutiliza las posiciones liberadas al frente sin desplazar el resto de elementos; una cola de doble extremo (*deque*) permite insertar y extraer tanto por el frente como por el final.

En una cola circular sobre un array de tamaño $m = 5$, tras encolar 5 elementos (índices 0-4) y desencolar 2 (frente avanza a 2), encolar un sexto elemento lo coloca en el índice $(4 + 1) \bmod 5 = 0$ —el hueco liberado por el primer desencolado— gracias a la aritmética modular, sin desplazar ningún elemento existente ni fallar por desbordamiento del array subyacente.

Se reproduce con `verification/chapter_results.sh circular_queue_wrap 5`.

```
ALGORITMO: Cola_Circular_Buffer_Lecturas
ESTRUCTURA DE DATOS:
- Buffer: Array estatico de tamano fijo K
- head: Indice del primer elemento a desencolar (inicialmente 0)
- tail: Indice de la siguiente posicion libre de insercion (inicialmente 0)
- count: Numero de elementos activos en el buffer (inicialmente 0)
COMPLEJIDAD: Enqueue O(1) tiempo, Dequeue O(1) tiempo, Espacio total O(K)

OPERACION ENQUEUE(Lectura_Fastq):
  Si count == K entonces:
    Retornar ERROR_BUFFER_OVERFLOW (El buffer esta completamente saturado)

  Buffer[tail] <- Lectura_Fastq
  tail <- (tail + 1) mod K      # Aritmetica modular: da la vuelta al inicio si tail = K
  count <- count + 1
  Retornar EXITO

OPERACION DEQUEUE():
  Si count == 0 entonces:
    Retornar ERROR_BUFFER_UNDERFLOW (El buffer esta vacio)

  Elemento_Extraido <- Buffer[head]
  head <- (head + 1) mod K      # Avanza el frente en tiempo O(1) sin desplazar celdas
  count <- count - 1
  Retornar Elemento_Extraido
```

7.2 Anticipo: Colas de prioridad

Cuando el orden de procesamiento no depende del orden de llegada sino de una prioridad biológica (por ejemplo, procesar antes las lecturas con mayor calidad Phred o los alineamientos con mayor puntuación), se utiliza una **cola de prioridad**. Si se implementara con una lista enlazada no ordenada, extraer el máximo exigiría un escaneo lineal $O(n)$ en cada paso. Esta limitación motivará el estudio de las estructuras arborescentes del capítulo siguiente.

Extraer repetidamente el elemento de máxima prioridad de una colección guardada en una lista enlazada simple sin ordenar exige recorrerla entera cada vez para encontrar el máximo: $O(n)$ por extracción, no $O(1)$.

Se reproduce con `verification/chapter_results.sh priority_scan_cost 100`.

8 La operación peligrosa: mutación concurrente e índices inválidos

Una de las fuentes más críticas de fallos en bioinformática ocurre al **modificar o redimensionar una estructura de datos mientras se está iterando sobre ella**. En almacenamiento contiguo, insertar o borrar en un bucle desplaza los índices restantes, provocando que se omitan elementos o se procesen duplicados. En almacenamiento enlazado, reasignar punteros sin variables temporales destruye la cadena de acceso a los datos restantes.

Protocolo preventivo en 3 pasos:

1. **Inmutabilidad durante el recorrido:** si se debe filtrar una colección, construir una nueva estructura con los elementos seleccionados en lugar de mutar la original durante la iteración.
2. **Validación rigurosa de límites:** verificar siempre que los índices $0 \leq i < n$ antes de cualquier acceso por posición.
3. **Preservación de punteros temporales:** al manipular nodos enlazados, guardar siempre una referencia al nodo siguiente antes de desacoplar un elemento.

9 Actividad de depuración: diagnóstico de errores en punteros

Analice el siguiente fragmento para eliminar un nodo en una lista enlazada con tres errores comunes:

```
# Lista simplemente enlazada: cada nodo tiene solo .dato y .siguiente
eliminarNodo(L, x):
  actual <- L.cabeza
  mientras actual.dato != x:      # Fallo 1
    actual <- actual.siguiente    # Fallo 2
  L.cabeza <- actual.siguiente    # Fallo 3
```

Tarea. Corrija los tres fallos y diga cuál provoca un fallo inmediato y visible, y cuáles producen un resultado incorrecto en silencio. Fíjese en que la lista es simplemente enlazada: los nodos no guardan a su antecesor.

Diagnóstico.

1. El bucle lee `actual.dato` sin haber comprobado que `actual` exista. Con una lista vacía falla en la primera vuelta; con un valor que no está en la lista, falla al llegar al final. Hay que comprobar antes de entrar y en cada vuelta que `actual` no sea nulo.
2. Avanza sin conservar el nodo previo, que es precisamente el que habrá que reenlazar. Sin él no se puede borrar nada que no sea la cabeza. Se arregla con una segunda variable que vaya un paso por detrás.
3. Reenlaza la cabeza siempre, encuentre el elemento donde lo encuentre. Si el nodo buscado está en medio, esta línea descabeza la lista y pierde todo lo anterior sin dar ningún error: la lista queda más corta y el programa sigue. Lo correcto es `previo.siguiente <- actual.siguiente`, y solo tocar la cabeza cuando el nodo encontrado sea la cabeza.

Atención. Los dos primeros fallos rompen el programa y se descubren pronto. El tercero devuelve una lista perfectamente válida, solo que equivocada, y puede sobrevivir meses en producción. Al revisar código sobre estructuras enlazadas, el caso que hay que probar primero no es el habitual: es borrar el primer elemento, borrar el último y borrar uno que no está.

10 Síntesis operativa y matriz comparativa de costes

REFERENCIA

Operación	Array	Lista Enlazada	Pila (LIFO)	Cola (FIFO)
Acceso por índice	$O(1)$ directo	$O(n)$ secuencial	—	—
Búsqueda no ordenada	$O(n)$	$O(n)$	—	—
Inserción al inicio	$O(n)$ (desplazamiento)	$O(1)$ (dos punteros)	—	—
Inserción al final	$O(1)$ amortizado	$O(1)$ (con cola)	push: $O(1)$	enqueue: $O(1)$
Borrado al inicio	$O(n)$ (desplazamiento)	$O(1)$	pop: $O(1)$	dequeue: $O(1)$
Localidad de caché	Óptima	Pobre	Variable	Variable

El almacenamiento contiguo (array) regala acceso por índice en $O(1)$ pero penaliza la inserción intermedia; el enlazado (lista, y con él pila y cola) regala inserción/borrado en $O(1)$ por los extremos pero penaliza el acceso. Elegir estructura es elegir qué operación se quiere que sea barata.

10.1 Tabla de diagnóstico de fallos en estructuras lineales

Síntoma observado	Causa probable	Comprobación y corrección
Bucle infinito al recorrer lista	Punteros enlazados en ciclo por mala inserción	Revisar que <code>nuevo.siguiente</code> se asigne antes que cabeza
El programa devuelve datos basura o falla	Acceso a índice fuera de límites ($i \geq n$)	Validar que $0 \leq i < n$ antes de consultar el array
Error al desencolar en buffer circular	Punteros no avanzan con módulo	Aplicar $(idx + 1) \bmod M$ en inserción y extracción
Condición $0.1 + 0.2 == 0.3$ da falsa	Imprecisión de coma flotante IEEE 754	Comparar mediante tolerancia $ a - b < \varepsilon$

10.2 Tabla de síntesis con preguntas de control

Necesidad del pipeline	Estructura recomendada	Pregunta de control
Acceso aleatorio masivo por posición	Array / Vector contiguo	¿El tamaño es fijo o se conoce de antemano?
Inserciones continuas al inicio/final	Lista enlazada con puntero de cola	¿Necesito recorrer hacia atrás (doble enlace)?
Validación de anidamientos / Deshacer	Pila (<i>Stack</i> LIFO)	¿He controlado la pila vacía antes de pop?
Procesamiento por orden de llegada	Cola (<i>Queue</i> FIFO circular)	¿El buffer de anillo gestiona el <i>wrap-around</i> ?

11 Glosario conceptual

REFERENCIA

- **Tipo de Dato Abstracto (TDA):** especificación funcional de una estructura sin fijar su implementación.
- **Almacenamiento contiguo:** disposición consecutiva en memoria con cálculo directo de dirección $O(1)$.
- **Almacenamiento enlazado:** disposición dispersa de nodos conectados mediante punteros explícitos.
- **Array dinámico:** vector contiguo con redimensionamiento geométrico ($O(1)$ amortizado).
- **Pila (Stack):** estructura LIFO con acceso exclusivo por la cima.
- **Cola (Queue):** estructura FIFO con inserción al final y extracción por el frente.
- **Buffer circular:** cola sobre array estático con punteros modulares.
- **Complemento a dos:** codificación binaria de enteros con signo.
- **IEEE 754:** estándar de representación de números en coma flotante.
- **Localidad de caché:** eficiencia de transferencia entre memoria RAM y CPU por contigüidad espacial.

12 Conexión con el curso y siguientes pasos

Las estructuras lineales estudiadas en este capítulo son los bloques constructivos sobre los que se desarrollan los temas posteriores:

- El capítulo siguiente da el salto a las estructuras no lineales: árboles binarios de búsqueda, montículos, tablas de dispersión y grafos de interacción biológica.
- El de búsqueda y ordenación compara la eficiencia de lo contiguo y lo enlazado sobre algoritmos concretos.
- Los de probabilidad y modelos de Markov representan vectores de probabilidad y matrices de transición.

- El de alineamiento usa matrices bidimensionales para programación dinámica, con Needleman–Wunsch y Smith–Waterman.
- El de aprendizaje automático estructura tensores y grafos de cómputo.

13 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de estructuras de datos de la asignatura, escrita por Álvaro Serrano Navarro. Los costes de las operaciones sobre cada estructura y el análisis amortizado del array dinámico siguen a Cormen y colaboradores [3]; los ejemplos de aplicación biológica, a Compeau y Pevzner [2]; y las implementaciones sobre arrays de Shell, a Allesina y Wilmes [1].

Todos los resultados numéricos se reproducen con `verification/chapter_results.sh` tal como están impresos.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos contrastivos de estructuras de datos; no sostiene ninguna afirmación en solitario.
- [2] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático del array como representación de secuencias y de la pila en el emparejamiento de bases del ARN.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para el análisis amortizado de arrays dinámicos y el coste de las estructuras lineales; no se reproduce, solo se cita.